

My Project

Generated by Doxygen 1.14.0

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 Studentas Class Reference	7
4.1.1 Detailed Description	9
4.1.2 Constructor & Destructor Documentation	9
4.1.2.1 Studentas() [1/3]	9
4.1.2.2 Studentas() [2/3]	9
4.1.2.3 Studentas() [3/3]	10
4.1.3 Member Function Documentation	10
4.1.3.1 egzaminas()	10
4.1.3.2 galutinis()	10
4.1.3.3 galutinisMed()	10
4.1.3.4 galutinisVid()	11
4.1.3.5 info()	11
4.1.3.6 nd()	11
4.1.3.7 operator<()	11
4.1.3.8 operator=() [1/2]	11
4.1.3.9 operator=() [2/2]	12
4.1.3.10 pridetiNd()	12
4.1.3.11 setEgzaminas()	12
4.1.3.12 setNd()	13
4.1.4 Friends And Related Symbol Documentation	13
4.1.4.1 operator<<	13
4.1.4.2 operator>>	13
4.2 Zmogus Class Reference	14
4.2.1 Detailed Description	15
4.2.2 Constructor & Destructor Documentation	15
4.2.2.1 Zmogus() [1/3]	15
4.2.2.2 Zmogus() [2/3]	15
4.2.2.3 Zmogus() [3/3]	15
4.2.3 Member Function Documentation	16
4.2.3.1 info()	16
4.2.3.2 operator=() [1/2]	16
4.2.3.3 operator=() [2/2]	16
4.2.3.4 pavarde()	16

4.2.3.5 setPavarde()	16
4.2.3.6 setVardas()	17
4.2.3.7 vardas()	17
5 File Documentation	19
5.1 Filegen.h File Reference	19
5.1.1 Detailed Description	19
5.1.2 Function Documentation	19
5.1.2.1 generuotiFaila()	19
5.2 Filegen.h	20
5.3 Fileproc.h	20
5.4 Lib.h File Reference	20
5.4.1 Detailed Description	21
5.4.2 Function Documentation	21
5.4.2.1 generuotiDuomenis()	21
5.4.2.2 isvestilFailus()	21
5.4.2.3 iverstiDuomenis()	21
5.4.2.4 nuskaitytiFaila()	22
5.4.2.5 padalintiStudentus()	22
5.4.2.6 spausdintiDuomenis()	22
5.5 Lib.h	23
5.6 student.h File Reference	23
5.7 student.h	24
5.8 Timer.h File Reference	25
5.8.1 Detailed Description	25
5.8.2 Function Documentation	25
5.8.2.1 elapsed_time()	25
5.8.2.2 start_timer()	26
5.9 Timer.h	26
5.10 utils.h File Reference	26
5.10.1 Detailed Description	27
5.10.2 Function Documentation	27
5.10.2.1 rusiavimas() [1/2]	27
5.10.2.2 rusiavimas() [2/2]	27
5.10.2.3 skaityti()	27
5.10.2.4 skirstymas1()	28
5.10.2.5 skirstymas2()	28
5.10.2.6 skirstymas3() [1/2]	29
5.10.2.7 skirstymas3() [2/2]	29
5.11 utils.h	29
5.12 Zmogus.h File Reference	30
5.12.1 Detailed Description	31

5.13 Zmogus.h	31
Index	33

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Zmogus	14
Studentas	7

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

[Studentas](#)

Klasė, kuri aprašo studentą, paveldinti iš [Zmogus](#) klasės, saugo nd, egzamina, skaičiuoja rezultatus [7](#)

[Zmogus](#)

Abstrakti bazinė klasė, kuri aprašo žmogaus duomenis (vardą ir pavardę) [14](#)

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

Filegen.h	Deklaruoja funkciją testinių duomenų failams generuoti	19
Fileproc.h		20
Lib.h	Bendroji biblioteka ir funkcijos darbui su Studentas duomenimis ir pagalbiniai apibrėžimai . . .	20
student.h		23
Timer.h	Pagalbinės funkcijos laiko matavimui (aukštos raiškos laikrodis)	25
utils.h	Pagalbinės funkcijos skirtos studentų duomenų apdorojimui	26
Zmogus.h	Apibrėžia abstrakčią bazinę klasę Žmogus, skirtą paveldėjimui	30

Chapter 4

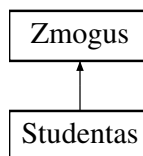
Class Documentation

4.1 Studentas Class Reference

Klasė, kuri aprašo studentą, paveldinti iš [Zmogus](#) klasės, saugo nd, egzamina, skaičiuoja rezultatus.

```
#include <student.h>
```

Inheritance diagram for Studentas:



Public Member Functions

- **Studentas** ()
Konstruktorius be parametru
- **Studentas** (const std::string &vard, const std::string &pav, const std::vector< int > &nd, int egz)
Konstruktorius su parametrais.
- **Studentas** (const **Studentas** &other)
kopijavimo konstruktorius
- **Studentas** & operator= (const **Studentas** &other)
kopijavimo priskyrimo operatorius
- **Studentas** (**Studentas** &&other) noexcept
perkėlimo konstruktorius
- **Studentas** & operator= (**Studentas** &&other) noexcept
perkėlimo priskyrimo operatorius
- **~Studentas** ()
Destruktorius.
- std::vector< int > nd () const
Grąžina studento namų darbų pažymių vektorių
- int egzaminas () const
grąžina studento egzaminą pažymį
- double galutinisVid () const

- gražina studento galutinį pažymį pagal vidurkį*
- double **galutinisMed** () const
- gražina studento galutinį pažymį pagal medianą*
- double **galutinis** () const
- gražina studento galutinį pažymį*
- void **setNd** (const std::vector< int > &n)
- nustato naują namų darbų pažymių vektorių*
- void **setEgzaminas** (int e)
- nustato naują egzamino pažymį*
- void **skaiciuoti_vidurki_ir_mediana** ()
- suskaiciuoja vidurkį ir medianą iš namų darbų pažymių*
- void **skaiciuoti_galutini** ()
- suskaiciuoja galutinį pažymį pagal vidurkį ir egzaminą*
- void **isvalytiNd** ()
- išvalo namų darbų pažymių vektorių, visus elementus ištrina*
- void **pridetiNd** (int n)
- prideja naują namų darbų pažymį į vektorių*
- bool **operator<** (const **Studentas** &other) const
- operatorius, kuris lygina du studentus pagal galutinį pažymį*
- void **info** () const override
- realizuoja abstraktų metodą iš **Zmogus** klasės*

Public Member Functions inherited from **Zmogus**

- **Zmogus** ()
- Konstruktorius be parametų, inicializuoja vardą ir pavardę tuščiais stringais.*
- **Zmogus** (const std::string &v, const std::string &p)
- Konstruktorius su parametrais, inicializuoja vardą ir pavardę*
- **Zmogus** (const **Zmogus** &other)
- kopijavimo konstruktorius*
- **Zmogus** (**Zmogus** &&other) noexcept
- perkeliamasis konstruktorius*
- **Zmogus** & **operator=** (const **Zmogus** &other)
- kopijavimo priskyrimo operatorius*
- **Zmogus** & **operator=** (**Zmogus** &&other) noexcept
- perkėlimo priskyrimo operatorius*
- virtual ~**Zmogus** ()=default
- Destruktorius.*
- std::string **vardas** () const
- gražina žmogaus vardą*
- std::string **pavarde** () const
- gražina žmogaus pavardę*
- void **setVardas** (const std::string &v)
- nustato naują vardą*
- void **setPavarde** (const std::string &p)
- nustato naują pavardę*

Friends

- `std::istream & operator>>` (`std::istream &is`, `Studentas &s`)
Ivesties operatorius.
- `std::ostream & operator<<` (`std::ostream &os`, `const Studentas &s`)
Išvesties operatorius.

Additional Inherited Members

Protected Attributes inherited from `Zmogus`

- `std::string vardas_`
Žmogaus vardas.
- `std::string pavarde_`
Žmogaus pavardė

4.1.1 Detailed Description

Klasė, kuri aprašo studentą, paveldinti iš `Zmogus` klasės, saugo `nd`, egzamina, skaičiuoja rezultatus.

4.1.2 Constructor & Destructor Documentation

4.1.2.1 `Studentas()` [1/3]

```
Studentas::Studentas (
    const std::string & vard,
    const std::string & pav,
    const std::vector< int > & nd,
    int egz)
```

Konstruktorius su parametrais.

Parameters

<i>vard</i>	studento vardas
<i>pav</i>	studento pavardė
<i>nd</i>	studento namų darbų pažymiai
<i>egz</i>	studento egzaminas pažymys

4.1.2.2 `Studentas()` [2/3]

```
Studentas::Studentas (
    const Studentas & other)
```

kopijavimo konstruktorius

Naudojamas, kai reikia sukurti naują `Studentas` objektą, kuris yra kopija kito `Studentas` objekto.

Parameters

<i>other</i>	kitas Studentas objektas
--------------	--

4.1.2.3 Studentas() [3/3]

```
Studentas::Studentas (  
    Studentas && other) [noexcept]
```

perkėlimo konstruktorius

Naudojamas, kai reikia sukurti naują [Studentas](#) objektą, perkeliant iš kitų [Studentas](#) objekto.

Parameters

<i>other</i>	kitas Studentas objektas
--------------	--

4.1.3 Member Function Documentation**4.1.3.1 egzaminas()**

```
int Studentas::egzaminas () const [inline]
```

gražina studento egzaminą pažymį

Returns

egzamino pažymys

4.1.3.2 galutinis()

```
double Studentas::galutinis () const [inline]
```

gražina studento galutinį pažymį

Returns

galutinis pažymys

Galutinis pažymys yra apskaičiuojamas pagal formulę: $0.4 * \text{vidurkis} + 0.6 * \text{egzaminas}$

4.1.3.3 galutinisMed()

```
double Studentas::galutinisMed () const [inline]
```

gražina studento galutinį pažymį pagal medianą

Returns

galutinis pažymys pagal medianą

4.1.3.4 galutinisVid()

```
double Studentas::galutinisVid () const [inline]
```

grąžina studento galutinį pažymį pagal vidurkį

Returns

galutinis pažymys pagal vidurkį

4.1.3.5 info()

```
void Studentas::info () const [inline], [override], [virtual]
```

realizuoja abstraktų metodą iš [Zmogus](#) klasės

Implements [Zmogus](#).

4.1.3.6 nd()

```
std::vector< int > Studentas::nd () const [inline]
```

Grąžina studento namų darbų pažymių vektorių

Returns

namų darbų pažymiai

4.1.3.7 operator<()

```
bool Studentas::operator< (  
    const Studentas & other) const [inline]
```

operatorius, kuris lygina du studentus pagal galutinį pažymį

Parameters

<i>other</i>	kitas studentas objektas
--------------	--------------------------

4.1.3.8 operator=() [1/2]

```
Studentas & Studentas::operator= (  
    const Studentas & other)
```

kopijavimo priskyrimo operatorius

Parameters

<i>other</i>	kitas Studentas objektas
--------------	--

Naudojamas, kai reikia priskirti vieną [Studentas](#) objektą kitam.

Returns

[Studentas](#)& nuoroda į šį objektą

Note

Šis operatorius yra svarbus, kad būtų galima teisingai tvarkyti išteklius, pvz., dinamiškai paskirtą atmintį.

4.1.3.9 operator=() [2/2]

```
Studentas & Studentas::operator= (  
    Studentas && other) [noexcept]
```

perkėlimo priskyrimo operatorius

Parameters

<i>other</i>	kitas Studentas objektas
--------------	--

Naudojamas, kai reikia priskirti vieną [Studentas](#) objektą kitam, perkeliant išteklius.

4.1.3.10 pridetiNd()

```
void Studentas::pridetiNd (  
    int n) [inline]
```

prideja naują namų darbų pažymį į vektorių

Parameters

<i>n</i>	pridedamas pažymys
----------	--------------------

4.1.3.11 setEgzaminas()

```
void Studentas::setEgzaminas (  
    int e) [inline]
```

nustato naują egzamino pažymį

Parameters

<i>e</i>	egzaminas pažymys
----------	-------------------

4.1.3.12 setNd()

```
void Studentas::setNd (
    const std::vector< int > & n) [inline]
```

nustato nauja namų darbų pažymių vektorių

Parameters

<i>n</i>	namų darbų pažymiai
----------	---------------------

4.1.4 Friends And Related Symbol Documentation**4.1.4.1 operator<<**

```
std::ostream & operator<< (
    std::ostream & os,
    const Studentas & s) [friend]
```

Išvesties operatorius.

Parameters

<i>os</i>	išvesties srautas
<i>s</i>	studentas objektas

Naudojamas studento duomenims išvesti į srautą.

4.1.4.2 operator>>

```
std::istream & operator>> (
    std::istream & is,
    Studentas & s) [friend]
```

Įvesties operatorius.

Parameters

<i>is</i>	įvesties srautas
<i>s</i>	studentas objektas

Naudojamas studento duomenims įvesti iš srauto.

The documentation for this class was generated from the following files:

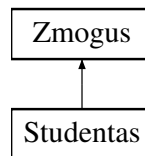
- [student.h](#)
- [student.cpp](#)

4.2 Zmogus Class Reference

abstrakti bazinė klasė, kuri aprašo žmogaus duomenis (vardą ir pavardę)

```
#include <Zmogus.h>
```

Inheritance diagram for Zmogus:



Public Member Functions

- **Zmogus ()**
Konstruktorius be parametų, inicializuoja vardą ir pavardę tuščiais stringais.
- **Zmogus** (const std::string &v, const std::string &p)
Konstruktorius su parametrais, inicializuoja vardą ir pavardę
- **Zmogus** (const **Zmogus** &other)
kopijavimo konstruktorius
- **Zmogus** (**Zmogus** &&other) noexcept
perkeliamasis konstruktorius
- **Zmogus** & **operator=** (const **Zmogus** &other)
kopijavimo priskyrimo operatorius
- **Zmogus** & **operator=** (**Zmogus** &&other) noexcept
perkėlimo priskyrimo operatorius
- virtual ~**Zmogus** ()=default
Destruktorius.
- std::string **vardas** () const
grąžina žmogaus vardą
- std::string **pavarde** () const
grąžina žmogaus pavardę
- void **setVardas** (const std::string &v)
nustato naują vardą
- void **setPavarde** (const std::string &p)
nustato naują pavardę
- virtual void **info** () const =0
grąžina žmogaus informaciją

Protected Attributes

- std::string **vardas_**
Žmogaus vardas.
- std::string **pavarde_**
Žmogaus pavardė

4.2.1 Detailed Description

abstrakti bazinė klasė, kuri aprašo žmogaus duomenis (vardą ir pavardę)

4.2.2 Constructor & Destructor Documentation

4.2.2.1 Zmogus() [1/3]

```
Zmogus::Zmogus (
    const std::string & v,
    const std::string & p) [inline]
```

Konstruktorius su parametrais, inicializuoja vardą ir pavardę

Parameters

<i>v</i>	Žmogaus vardas
<i>p</i>	Žmogaus pavardė

4.2.2.2 Zmogus() [2/3]

```
Zmogus::Zmogus (
    const Zmogus & other) [inline]
```

kopijavimo konstruktorius

Parameters

<i>other</i>	kitas Zmogus objektas
--------------	---------------------------------------

4.2.2.3 Zmogus() [3/3]

```
Zmogus::Zmogus (
    Zmogus && other) [inline], [noexcept]
```

perkeliamasis konstruktorius

Parameters

<i>other</i>	kitas Zmogus objektas
--------------	---------------------------------------

4.2.3 Member Function Documentation

4.2.3.1 info()

```
virtual void Zmogus::info () const [pure virtual]
```

grąžina žmogaus informaciją

Šis metodas yra grynasis virtualus metodas, todėl kiekviena paveldėta klasė turi jį įgyvendinti.

Implemented in [Studentas](#).

4.2.3.2 operator=() [1/2]

```
Zmogus & Zmogus::operator= (
    const Zmogus & other) [inline]
```

kopijavimo priskyrimo operatorius

Parameters

<i>other</i>	kitas Zmogus objektas
--------------	---------------------------------------

4.2.3.3 operator=() [2/2]

```
Zmogus & Zmogus::operator= (
    Zmogus && other) [inline], [noexcept]
```

perkėlimo priskyrimo operatorius

Parameters

<i>other</i>	kitas Zmogus objektas
--------------	---------------------------------------

4.2.3.4 pavarde()

```
std::string Zmogus::pavarde () const [inline]
```

grąžina žmogaus pavardę

Returns

pavarde

4.2.3.5 setPavarde()

```
void Zmogus::setPavarde (
    const std::string & p) [inline]
```

nustato naują pavardę

Parameters

<i>p</i>	nauja pavardė
----------	---------------

4.2.3.6 setVardas()

```
void Zmogus::setVardas (
    const std::string & v) [inline]
```

nustato naują vardą

Parameters

<i>v</i>	naujas vardas
----------	---------------

4.2.3.7 vardas()

```
std::string Zmogus::vardas () const [inline]
```

grąžina žmogaus vardą

Returns

vardas

The documentation for this class was generated from the following file:

- [Zmogus.h](#)

Chapter 5

File Documentation

5.1 Filegen.h File Reference

Deklaruoja funkciją testinių duomenų failams generuoti.

```
#include <string>
```

Functions

- void [generuotiFaila](#) (const std::string &pavadinimas, int irasuSkaicius, int pazymiuKiekis)
Generuoja studentų duomenų failą su nurodytu įrašų ir pažymių kiekiu.

5.1.1 Detailed Description

Deklaruoja funkciją testinių duomenų failams generuoti.

5.1.2 Function Documentation

5.1.2.1 generuotiFaila()

```
void generuotiFaila (  
    const std::string & pavadinimas,  
    int irasuSkaicius,  
    int pazymiuKiekis)
```

Generuoja studentų duomenų failą su nurodytu įrašų ir pažymių kiekiu.

Parameters

<i>pavadinimas</i>	Sukuriamo failo pavadinimas
<i>irasuSkaicius</i>	Studentų įrašų skaičius faile
<i>pazymiuKiekis</i>	Kiek pažymių turi kiekvienas studentas

5.2 Filegen.h

[Go to the documentation of this file.](#)

```
00001
00005 #ifndef FILEGEN_H
00006 #define FILEGEN_H
00007 #include <string>
00008
00015 void generuotiFaila(const std::string& pavadinimas, int irasuSkaicius, int pazymiuKiekis);
00016
00017 #endif
```

5.3 Fileproc.h

```
00001
00002 #ifndef FILEPROC_H
00003 #define FILEPROC_H
00004
00005 #include "Lib.h"
00006
00007
00008 void nuskaitytiFaila(const string& pav, vector<Studentas>& studentai);
00009
00010 void padalintiStudentus(const vector<Studentas>& studentai, vector<Studentas>& vargs,
    vector<Studentas>& kiet);
00011
00012 void isvestiIFailus(const vector<Studentas>& vargs, const vector<Studentas>& kiet, const string&
    pavPre);
00013
00014 #endif
```

5.4 Lib.h File Reference

Bendroji biblioteka ir funkcijos darbui su [Studentas](#) duomenimis ir pagalbiniai apibrėžimai.

```
#include <string>
#include <vector>
#include <iostream>
#include <fstream>
#include <sstream>
#include <numeric>
#include <iomanip>
#include <algorithm>
#include <random>
#include <chrono>
#include <deque>
#include <list>
#include "student.h"
```

Functions

- void [nuskaitytiFaila](#) (const string &pav, vector< [Studentas](#) > &studentai)
Nuskaityti studentų duomenis iš failo į vektorių
- void [padalintiStudentus](#) (const vector< [Studentas](#) > &studentai, vector< [Studentas](#) > &vargs, vector< [Studentas](#) > &kiet)
Padalinti studentus į dvi grupes pagal glautini pažymį
- void [isvestiIFailus](#) (const vector< [Studentas](#) > &vargs, const vector< [Studentas](#) > &kiet, const string &pav↵
Pre)

- Išveda studentų duomenis į failus (vargsiukai ir kietekai)*
- void [ivestiDuomenis](#) ([Studentas](#) &studentas)
- Įveda studento duomenis iš standartinio įvesties srauto.*
- void [generuotiDuomenis](#) ([Studentas](#) &studentas, int pazymiuKiekis)
- Generuoja studento duomenis su atsitiktiniais pažymiais.*
- void [spausdintiDuomenis](#) (const [Studentas](#) &studentas, int pasirinkimas)
- Išveda studento duomenis į standartinį išvesties srautą*

5.4.1 Detailed Description

Bendroji biblioteka ir funkcijos darbui su [Studentas](#) duomenimis ir pagalbinių apibrėžimai.

5.4.2 Function Documentation

5.4.2.1 generuotiDuomenis()

```
void generuotiDuomenis (
    Studentas & studentas,
    int pazymiuKiekis)
```

Generuoja studento duomenis su atsitiktiniais pažymiais.

Parameters

<i>studentas</i>	Studentas objektas, kuriam bus priskirti duomenys
<i>pazymiuKiekis</i>	Kiek pažymių generuoti (namų darbų)

5.4.2.2 isvestiIFailus()

```
void isvestiIFailus (
    const vector< Studentas > & vargs,
    const vector< Studentas > & kiet,
    const string & pavPre)
```

Išveda studentų duomenis į failus (vargsiukai ir kietekai)

Parameters

<i>vargs</i>	Vektorius su vargšų studentų duomenimis
<i>kiet</i>	Vektorius su kietų studentų duomenimis
<i>pavPre</i>	Pagrindinis failo pavadinimas (be priedų)

5.4.2.3 ivestiDuomenis()

```
void ivestiDuomenis (
    Studentas & studentas)
```

Įveda studento duomenis iš standartinio įvesties srauto.

Parameters

<i>studentas</i>	<code>Studentas</code> objektas, kuriam bus priskirti duomenys
------------------	--

5.4.2.4 nuskaitytiFaila()

```
void nuskaitytiFaila (
    const string & pav,
    vector< Studentas > & studentai)
```

Nuskaityti studentų duomenis iš failo į vektorių

Parameters

<i>pav</i>	Failo pavadinimas
<i>studentai</i>	Vektorius, į kurį bus įrašyti duomenys

5.4.2.5 padalintiStudentus()

```
void padalintiStudentus (
    const vector< Studentas > & studentai,
    vector< Studentas > & vargs,
    vector< Studentas > & kiet)
```

Padalina studentus į dvi grupes pagal glautini pažymį

Parameters

<i>studentai</i>	Vektorius su visais studentais
<i>vargs</i>	Vektorius, kuriame bus vargšų studentų duomenys
<i>kiet</i>	Vektorius, kuriame bus kietų studentų duomenys

5.4.2.6 spausdintiDuomenis()

```
void spausdintiDuomenis (
    const Studentas & studentas,
    int pasirinkimas)
```

Išveda studento duomenis į standartinį išvesties srautą

Parameters

<i>studentas</i>	<code>Studentas</code> objektas, kurio duomenys bus išvedami
<i>pasirinkimas</i>	0 - tik galutinis pažymys pagal medianą, 1 - tik pagal vidurkį, 2 - abu

5.5 Lib.h

[Go to the documentation of this file.](#)

```

00001
00005 #ifndef LIB_H
00006 #define LIB_H
00007
00008 #include <string>
00009 #include <vector>
00010 #include <iostream>
00011 #include <fstream>
00012 #include <sstream>
00013 #include <numeric>
00014 #include <iomanip>
00015 #include <algorithm>
00016 #include <random>
00017 #include <chrono>
00018 #include <deque>
00019 #include <list>
00020 #include "student.h"
00021
00022 using std::string;
00023 using std::vector;
00024 using std::list;
00025 using std::deque;
00026 using std::ifstream;
00027 using std::stringstream;
00028 using std::accumulate;
00029 using std::sort;
00030 using std::setw;
00031 using std::left;
00032 using std::fixed;
00033 using std::setprecision;
00034 using std::cout;
00035 using std::endl;
00036 using std::istream;
00037 using std::ostream;
00038 using std::getline;
00039 using std::invalid_argument;
00040 using std::runtime_error;
00041 using std::istreambuf_iterator;
00042 using std::to_string;
00043 using std::istringstream;
00044 using std::random_device;
00045 using std::mt19937;
00046 using std::uniform_int_distribution;
00047 using std::cin;
00048 using std::ofstream;
00049 using std::default_random_engine;
00050 using std::chrono::duration;
00051 using std::chrono::high_resolution_clock;
00052 using std::chrono::milliseconds;
00053 using std::chrono::seconds;
00054 using std::chrono::duration_cast;
00055 using std::chrono::system_clock;
00056 using std::chrono::time_point;
00057
00058
00059 // Funkcijos su klase Studentas
00065 void nuskaitytiFaila(const string& pav, vector<Studentas>& studentai);
00072 void padalintiStudentus(const vector<Studentas>& studentai, vector<Studentas>& vargs,
vector<Studentas>& kiet);
00079 void isvestiIfailus(const vector<Studentas>& vargs, const vector<Studentas>& kiet, const string&
pavPre);
00080
00081 // Pagalbinės funkcijos (jei reikia rankinio įvedimo/generavimo)
00086 void ivestiDuomenis(Studentas& studentas);
00092 void generuotiDuomenis(Studentas& studentas, int pazymiuKiekis);
00098 void spausdintiDuomenis(const Studentas& studentas, int pasirinkimas);
00099
00100 #endif

```

5.6 student.h File Reference

```

#include "Zmogus.h"
#include <string>
#include <vector>

```

```
#include <algorithm>
#include <numeric>
```

Classes

- class [Studentas](#)

Klasė, kuri aprašo studentą, paveldinti iš [Zmogus](#) klasės, saugo nd, egzamina, skaiciuoja rezultatus.

5.7 student.h

[Go to the documentation of this file.](#)

```
00001
00006 #ifndef STUDENT_H
00007 #define STUDENT_H
00008
00009 #include "Zmogus.h"
00010 #include <string>
00011 #include <vector>
00012 #include <algorithm>
00013 #include <numeric>
00014
00015 class Studentas :public Zmogus {
00016 private:
00017     //std::string vardas_;
00018     //std::string pavarde_;
00023     std::vector<int> nd_;
00028     int egzaminas_;
00033     double galutinisVid_;
00038     double galutinisMed_;
00043     double galutinis_;
00044
00045 public:
00046     // Konstruktoriai
00051     Studentas();
00059     Studentas(const std::string& vard, const std::string& pav, const std::vector<int>& nd, int egz);
00060
00061     // Kopijavimo konstruktorius ir operatorius
00067     Studentas(const Studentas& other); //Rule of five copy constructor
00075     Studentas& operator=(const Studentas& other); // Rule of five copy assignment operator
00081     Studentas(Studentas&& other) noexcept; // Rule of five move constructor
00087     Studentas& operator=(Studentas&& other) noexcept; // Rule of five move assignment operator
00088
00093     // Destruktorius
00094     ~Studentas();
00101
00102     // Įvesties operatorius (į rautą)
00103     friend std::istream& operator>>(std::istream& is, Studentas& s);
00104
00105     // Išvesties operatorius (iš rauto)
00112     friend std::ostream& operator<<(std::ostream& os, const Studentas& s);
00113
00114
00115     // Getteriai
00116     /*
00117     std::string vardas() const { return vardas_; }
00118     std::string pavarde() const { return pavarde_; }
00119     std::vector<int> nd() const { return nd_; }
00120     int egzaminas() const { return egzaminas_; }
00121     */
00122
00127     std::vector<int> nd() const { return nd_; } //zmogus.h
00133     int egzaminas() const { return egzaminas_; } //zmogus.h
00139     double galutinisVid() const { return galutinisVid_; }
00145     double galutinisMed() const { return galutinisMed_; }
00152     double galutinis() const { return galutinis_; }
00153
00154     // Setteriai
00155     /*
00156     void setVardas(const std::string& v) { vardas_ = v; }
00157     void setPavarde(const std::string& p) { pavarde_ = p; }
00158     */
00159
00164     void setNd(const std::vector<int>& n) { nd_ = n; }
00170     void setEgzaminas(int e) { egzaminas_ = e; }
```

```

00171
00172 // Kiti metodai
00173 void skaiciuoti_vidurki_ir_mediana();
00182 void skaiciuoti_galutini();
00187 void isvalytiNd() { nd_.clear(); }
00193 void pridetiNd(int n) { nd_.push_back(n); }
00194
00200 bool operator<(const Studentas& other) const { return galutinis_ < other.galutinis_; }
00201
00202 // Privalomas abstraktaus metodo realizavimas
00207 void info() const override {
00208     std::cout << "Studentas: " << vardas_ << " " << pavarde_ << std::endl;
00209 }
00210 };
00211
00212 #endif // STUDENT_H

```

5.8 Timer.h File Reference

Pagalbinės funkcijos laiko matavimui (aukštos raiškos laikrodis)

```
#include "Lib.h"
```

Typedefs

- using **TimerPoint** = high_resolution_clock::time_point
Aukštos raiškos laikrodžio laiko taškas (pradžiai ar pabaigai žymėti)

Functions

- **TimerPoint start_timer** ()
Gražina dabartinį laiką
- double **elapsed_time** (const **TimerPoint** &start)
Apskaičiuoja praėjusį laiką sekundėmis nuo pradžios taško.

5.8.1 Detailed Description

Pagalbinės funkcijos laiko matavimui (aukštos raiškos laikrodis)

5.8.2 Function Documentation

5.8.2.1 elapsed_time()

```
double elapsed_time (
    const TimerPoint & start) [inline]
```

Apskaičiuoja praėjusį laiką sekundėmis nuo pradžios taško.

Parameters

<i>start</i>	Pradžios laiko taškas
--------------	-----------------------

5.8.2.2 start_timer()

```
TimerPoint start_timer () [inline]
```

Gražina dabartinį laiką

Returns

`TimerPoint` - laiko žymė

5.9 Timer.h

[Go to the documentation of this file.](#)

```
00001
00005
00006 #ifndef TIMER_H
00007 #define TIMER_H
00008
00009 #include "Lib.h"
00010
00015 using TimerPoint = high_resolution_clock::time_point;
00016
00021 inline TimerPoint start_timer() {
00022     return high_resolution_clock::now();
00023 }
00024
00029 inline double elapsed_time(const TimerPoint& start) {
00030     return duration<double>(high_resolution_clock::now() - start).count();
00031 }
00032
00033 #endif
```

5.10 utils.h File Reference

pagalbinės funkcijos skirtos studentų duomenų apdorojimui

```
#include "Lib.h"
```

Enumerations

- enum `ContainerType` { `VECTOR` , `LIST` , `DEQUE` }
konteinerio tipų enumeracija (VECTOR, LIST, DEQUE)

Functions

- template<typename Container>
void `skaityti` (const std::string &filename, Container &students)
Skaito studentų duomenis iš failo į nurodytą konteinerį.
- template<typename Container>
void `rusiavimas` (Container &students)
rūšiuoja studentus pagal galutinį pažymį.
- template<> void `rusiavimas` (list< `Studentas` > &students)
Rūšiuoja studentų sąrašą pagal galutinį pažymį

- `template<typename Container>`
`void skirstymas1 (const Container &students, Container &vargs, Container &kiet)`
Skirsto studentus į dvi grupes: vargšus ir kietus.
- `template<typename Container>`
`void skirstymas2 (Container &students, Container &vargs)`
Skirsto studentus: vargšiai kopijuojami į naują, trinami iš bendro konteinerio.
- `template<typename Container>`
`void skirstymas3 (Container &students, Container &vargs)`
Skirsto studentus naudojant STL partition/remove_if.
- `template<> void skirstymas3 (list< Studentas > &students, list< Studentas > &vargs)`
Skirsto studentų sąrašą naudojant remove_if (specializacija list)

5.10.1 Detailed Description

pagalbinės funkcijos skirtos studentų duomenų apdorojimui

5.10.2 Function Documentation

5.10.2.1 rusiavimas() [1/2]

```
template<typename Container>
void rusiavimas (
    Container & students)
```

rūšiuoja studentus pagal galutinį pažymį.

Template Parameters

<i>Container</i>	STL, turintis begin() ir end()
------------------	--------------------------------

Parameters

<i>students</i>	studentų konteineris
-----------------	----------------------

5.10.2.2 rusiavimas() [2/2]

```
template<>
void rusiavimas (
    list< Studentas > & students) [inline]
```

Rūšiuoja studentų sąrašą pagal galutinį pažymį

Parameters

<i>students</i>	Studentų sąrašas (list).
-----------------	--------------------------

5.10.2.3 skaityti()

```
template<typename Container>
void skaityti (
    const std::string & filename,
    Container & students)
```

Skaito studentų duomenis iš failo į nurodytą konteinerį.

Template Parameters

<i>Container</i>	STL suderinamas konteineris
------------------	-----------------------------

Parameters

<i>filename</i>	Failo pavadinimas, iš kurio bus skaitomi studentų duomenys.
<i>students</i>	Konteineris, į kurį bus rašomi studentų duomenys.

5.10.2.4 skirstymas1()

```
template<typename Container>
void skirstymas1 (
    const Container & students,
    Container & vargs,
    Container & kiet)
```

Skirsto studentus į dvi grupes: vargšus ir kietus.

Template Parameters

<i>Container</i>	STL suderinamas konteineris
------------------	-----------------------------

Parameters

<i>students</i>	Originalus studentų konteineris.
<i>vargs</i>	Vargšų studentų konteineris, kuriame bus saugomi studentai su galutiniu pažymiu mažesniu nei 5.0.
<i>kiet</i>	Kietų studentų konteineris, kuriame bus saugomi studentai su galutiniu pažymiu 5.0 ar didesniu.

5.10.2.5 skirstymas2()

```
template<typename Container>
void skirstymas2 (
    Container & students,
    Container & vargs)
```

Skirsto studentus: vargšiškai kopijuojami į naują, trinami iš bendro konteinerio.

Template Parameters

<i>Container</i>	STL suderinamas konteineris
------------------	-----------------------------

Parameters

<i>students</i>	Pradinis studentų konteineris (bus iš jo trinami)
<i>vargs</i>	Konteineris, kuriame bus studentai su galutinis < 5

5.10.2.6 skirstymas3() [1/2]

```
template<typename Container>
void skirstymas3 (
    Container & students,
    Container & vars)

```

Skirsto studentus naudojant STL partition/remove_if.

Template Parameters

<i>Container</i>	STL suderinamas konteineris
------------------	-----------------------------

Parameters

<i>students</i>	Pradinis studentų konteineris (bus iš jo trinami)
<i>vars</i>	Konteineris, kuriame bus studentai su galutinis < 5

5.10.2.7 skirstymas3() [2/2]

```
template<>
void skirstymas3 (
    list< Studentas > & students,
    list< Studentas > & vars) [inline]

```

Skirsto studentų sąrašą naudojant remove_if (specializacija list)

Parameters

<i>students</i>	Studentų sąrašas (list), iš kurio bus trinami
<i>vars</i>	Sąrašas, į kurį bus kopijuojami studentai su galutinis < 5

5.11 utils.h

[Go to the documentation of this file.](#)

```
00001
00005 #ifndef UTILS_H
00006 #define UTILS_H
00007
00008 #include "Lib.h"
00009
00014 enum ContainerType { VECTOR, LIST, DEQUE };
00015
00016 // Nuskaitymo funkcija:
00023 template<typename Container>
00024 void skaityti(const std::string& filename, Container& students) {
00025     ifstream fin(filename);
00026     string header;
00027     getline(fin, header);
00028     string vardas, pavarde;
00029     while (fin » vardas » pavarde) {
00030         vector<int> nd;
00031         int n;
00032         for (int i = 0; i < 5; i++) {
00033             if (fin » n) nd.push_back(n);
00034         }

```

```

00035         int egzaminas;
00036         fin » egzaminas;
00037         Studentas s(vardas, pavarde, nd, egzaminas);
00038         s.skaiciuoti_galutini();
00039         students.push_back(s);
00040     }
00041 }
00042
00043 // Rūšiavimas
00050 template <typename Container>
00051 void rusiavimas(Container& students) {
00052     sort(students.begin(), students.end(),
00053         [](const Studentas& a, const Studentas& b) { return a.galutinis() < b.galutinis(); });
00054 }
00059 template <>
00060 inline void rusiavimas(list<Studentas>& students) {
00061     students.sort([](const Studentas& a, const Studentas& b) { return a.galutinis() < b.galutinis();
00062 });
00063 }
00064 // 1 strategija: Kopijuojama į du naujus konteinerius
00072 template <typename Container>
00073 void skirstymas1(const Container& students, Container& vargs, Container& kiet) {
00074     for (const auto& s : students) {
00075         if (s.galutinis() < 5.0) vargs.push_back(s);
00076         else kiet.push_back(s);
00077     }
00078 }
00085 // 2 strategija: Tik vargšiškai naujame, trinami iš bendro
00086 template <typename Container>
00087 void skirstymas2(Container& students, Container& vargs) {
00088     auto it = students.begin();
00089     while (it != students.end()) {
00090         if (it->galutinis() < 5.0) {
00091             vargs.push_back(*it);
00092             it = students.erase(it);
00093         } else ++it;
00094     }
00095 }
00102 // 3 strategija: STL partition/remove_if
00103 template <typename Container>
00104 void skirstymas3(Container& students, Container& vargs) {
00105     auto it = partition(students.begin(), students.end(), [](const Studentas& s) {
00106         return s.galutinis() < 5.0;
00107     });
00108     vargs.insert(vargs.end(), students.begin(), it);
00109     students.erase(students.begin(), it);
00110 }
00111
00117 // Specializacija list konteineriui
00118 template <>
00119 inline void skirstymas3(list<Studentas>& students, list<Studentas>& vargs) {
00120     students.remove_if([&vargs](const Studentas& s) {
00121         if (s.galutinis() < 5.0) {
00122             vargs.push_back(s);
00123             return true;
00124         }
00125         return false;
00126     });
00127 }
00128
00129 #endif

```

5.12 Zmogus.h File Reference

apibrėžia abstrakčią bazinę klasę Žmogus, skirtą paveldėjimui

```

#include <string>
#include <iostream>

```

Classes

- class [Zmogus](#)

abstakti bazinė klasė, kuri aprašo žmogaus duomenis (vardą ir pavardę)

5.12.1 Detailed Description

apibrėžia abstrakčią bazinę klasę Žmogus, skirtą paveldėjimui

5.13 Zmogus.h

[Go to the documentation of this file.](#)

```

00001
00005 #ifndef ZMOGUS_H
00006 #define ZMOGUS_H
00007
00008 #include <string>
00009 #include <iostream>
00010
00011 // Abstrakti bazinė klasė Žmogus
00017 class Zmogus {
00018 protected:
00023     std::string vardas_;
00028     std::string pavarde_;
00029 public:
00034     Zmogus() : vardas_ {}, pavarde_ {} {}
00041     Zmogus(const std::string& v, const std::string& p) : vardas_(v), pavarde_(p) {}
00047     Zmogus(const Zmogus& other) : vardas_(other.vardas_), pavarde_(other.pavarde_) {}
00053     Zmogus(Zmogus&& other) noexcept : vardas_(std::move(other.vardas_)),
    pavarde_(std::move(other.pavarde_)) {}
00059     Zmogus& operator=(const Zmogus& other) {
00060         if (this != &other) {
00061             vardas_ = other.vardas_;
00062             pavarde_ = other.pavarde_;
00063         }
00064         return *this;
00065     }
00071     Zmogus& operator=(Zmogus&& other) noexcept {
00072         if (this != &other) {
00073             vardas_ = std::move(other.vardas_);
00074             pavarde_ = std::move(other.pavarde_);
00075         }
00076         return *this;
00077     }
00082     virtual ~Zmogus() = default;
00083
00089     std::string vardas() const { return vardas_; }
00095     std::string pavarde() const { return pavarde_; }
00101     void setVardas(const std::string& v) { vardas_ = v; }
00107     void setPavarde(const std::string& p) { pavarde_ = p; }
00108
00109     // daro klasę abstrakčia
00115     virtual void info() const = 0;
00116 };
00117
00118 #endif

```


Index

egzaminas
 Studentas, 10
elapsed_time
 Timer.h, 25

Filegen.h, 19
 generuotiFaila, 19

galutinis
 Studentas, 10
galutinisMed
 Studentas, 10
galutinisVid
 Studentas, 10
generuotiDuomenis
 Lib.h, 21
generuotiFaila
 Filegen.h, 19

info
 Studentas, 11
 Zmogus, 16
isvestiIFailus
 Lib.h, 21
ivestiDuomenis
 Lib.h, 21

Lib.h, 20
 generuotiDuomenis, 21
 isvestiIFailus, 21
 ivestiDuomenis, 21
 nuskaitytiFaila, 22
 padalintiStudentus, 22
 spausdintiDuomenis, 22

nd
 Studentas, 11
nuskaitytiFaila
 Lib.h, 22

operator<
 Studentas, 11
operator<<
 Studentas, 13
operator>>
 Studentas, 13
operator=
 Studentas, 11, 12
 Zmogus, 16

padalintiStudentus
 Lib.h, 22

pavarde
 Zmogus, 16
pridetiNd
 Studentas, 12

rusiavimas
 utils.h, 27

setEgzaminas
 Studentas, 12
setNd
 Studentas, 13
setPavarde
 Zmogus, 16
setVardas
 Zmogus, 17
skaityti
 utils.h, 27
skirstymas1
 utils.h, 28
skirstymas2
 utils.h, 28
skirstymas3
 utils.h, 28, 29
spausdintiDuomenis
 Lib.h, 22
start_timer
 Timer.h, 25
student.h, 23
Studentas, 7
 egzaminas, 10
 galutinis, 10
 galutinisMed, 10
 galutinisVid, 10
 info, 11
 nd, 11
 operator<, 11
 operator<<, 13
 operator>>, 13
 operator=, 11, 12
 pridetiNd, 12
 setEgzaminas, 12
 setNd, 13
 Studentas, 9, 10

Timer.h, 25
 elapsed_time, 25
 start_timer, 25

utils.h, 26

- rusiavimas, [27](#)
- skaityti, [27](#)
- skirstymas1, [28](#)
- skirstymas2, [28](#)
- skirstymas3, [28](#), [29](#)

vardas

- Zmogus, [17](#)

Zmogus, [14](#)

- info, [16](#)
- operator=, [16](#)
- pavarde, [16](#)
- setPavarde, [16](#)
- setVardas, [17](#)
- vardas, [17](#)
- Zmogus, [15](#)

Zmogus.h, [30](#)